

CSA1717-ARTIFICIAL INTELLIGENCE

PRACTICAL LAB EXPERIMENTS

RAGAVI S – (192565027)

EXPERIMENT 7: DFS

Aim

To write a python program to implement DFS.

Objective

- To understand the concept of Depth-First Search (DFS) traversal.
- To implement DFS using recursion in Python.
- To display the DFS traversal order from a given starting vertex.

Algorithm

Step 1: Initialize a set to keep track of visited nodes.

Step 2: Call the DFS-Visit function with the starting node and the set of visited nodes.

Step 3: In the DFS-Visit function, add the current node to the set of visited nodes.

Step 4: For each neighbor of the current node, if it has not been visited yet, recursively call the DFS-Visit function with the neighbor node and the set of visited nodes.

Step 5: Repeat steps 3-4 until all reachable nodes have been visited.

Source Code

DFS Implementation in Python

```
def dfs(graph, vertex, visited=None):
```

```
    if visited is None:
```

```
        visited = set()
```

```
    visited.add(vertex)
```

```
    print(vertex, end=" ")
```

```

for neighbor in graph[vertex]:
    if neighbor not in visited:
        dfs(graph, neighbor, visited)

# Input
n = int(input("Enter number of vertices: "))

graph = {}

print("Enter adjacency list:")

for i in range(n):
    neighbors = list(map(int, input(f"Enter neighbors of vertex {i}: ").split()))
    graph[i] = neighbors

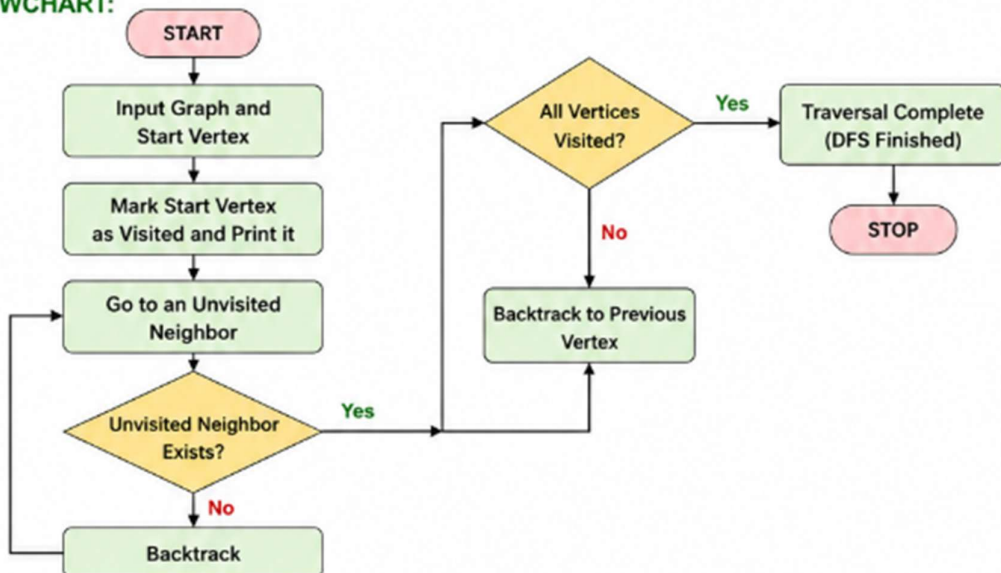
start = int(input("Enter starting vertex: "))

print("\nDFS Traversal Order:")
dfs(graph, start)

```

Flowchart

FLOWCHART:



Output

```
Enter number of vertices: 5
Enter adjacency list:
Enter neighbors of vertex 0: 1 2
Enter neighbors of vertex 1: 0 3 4
Enter neighbors of vertex 2: 0
Enter neighbors of vertex 3: 1
Enter neighbors of vertex 4: 1
Enter starting vertex: 0

DFS Traversal Order:
0 1 3 4 2
>>>|
```

Result

The Breadth-First Search (BFS) algorithm successfully traversed the graph level by level using a queue. All reachable vertices were visited in the correct BFS order without repetition.